C Loops

The looping can be defined as repeating the same process multiple times until a specific condition satisfies. There are three types of loops used in the C language. In this part of the tutorial, we are going to learn all the aspects of C loops.

Why use loops in C language?

The looping simplifies the complex problems into the easy ones. It enables us to alter the flow of the program so that instead of writing the same code again and again, we can repeat the same code for a finite number of times. For example, if we need to print the first 10 natural numbers then, instead of using the printf statement 10 times, we can print inside a loop which runs up to 10 iterations.

Advantage of loops in C

1) It provides code reusability.

2) Using loops, we do not need to write the same code again and again.

3) Using loops, we can traverse over the elements of data structures (array or linked lists).

Types of C Loops

There are three types of loops in C language that is given below:

1. do while
2. while
3. for

do-while loop in C

The do-while loop continues until a given condition satisfies. It is also called post tested loop. It is used when it is necessary to execute the loop at least once (mostly menu driven programs).

The syntax of do-while loop in c language is given below:

1. **do**{
2. //code to be executed
3. }**while**(condition);

The components are divided into the following:

- The ***do keyword*** marks the beginning of the Loop.
- The ***code block*** within ***curly braces {}*** is the body of the loop, which contains the code you want to repeat.

- The ***while keyword*** is followed by a condition enclosed in parentheses (). After the code block has been run, this condition is verified. If the condition is ***true***, the loop continues else, the ***loop ends***.

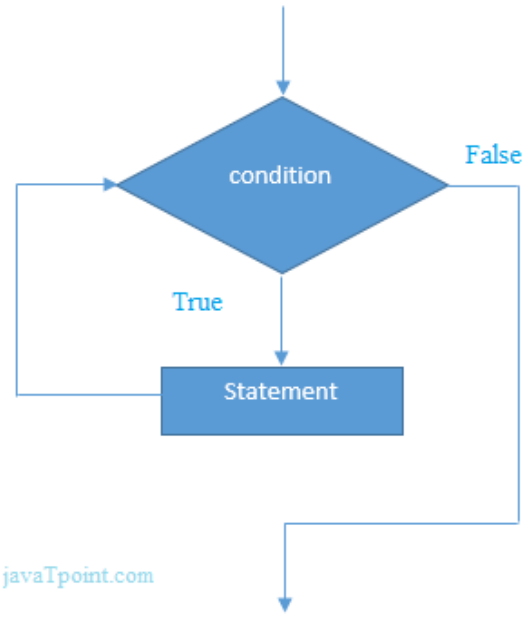
while loop in C

The while loop in c is to be used in the scenario where we don't know the number of iterations in advance. The block of statements is executed in the while loop until the condition specified in the while loop is satisfied. It is also called a pre-tested loop.

The syntax of while loop in c language is given below:

```
1. while(condition){
2. //code to be executed
3. }
```

Flowchart of while loop in C



Example of the while loop in C language

Let's see the simple program of while loop that prints 1 to 10.

```
1. #include<stdio.h>
```

```
2. int main(){
3. int i=1;
4. while(i<=10){
5. printf("%d \n",i);
6. i++;
7. }
8. return 0;
9. }
```

Output

1

2

3

4

5

6

7

8

9

10

Properties of while loop

- A conditional expression is used to check the condition. The statements defined inside the while loop will repeatedly execute until the given condition fails.
- The condition will be true if it returns 0. The condition will be false if it returns any non-zero number.
- In while loop, the condition expression is compulsory.
- Running a while loop without a body is possible.
- We can have more than one conditional expression in while loop.
- If the loop body contains only one statement, then the braces are optional.

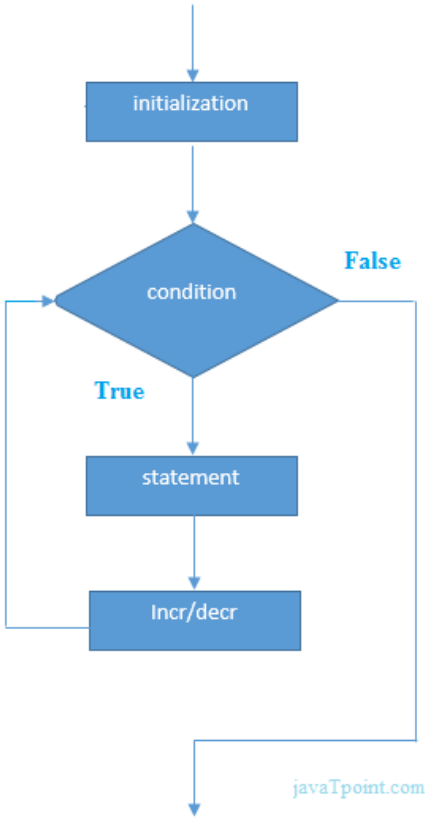
for loop in C

The for loop is used in the case where we need to execute some part of the code until the given condition is satisfied. The for loop is also called as a per-tested loop. It is better to use for loop if the number of iteration is known in advance.

The syntax of for loop in c language is given below:

1. **for**(initialization;condition;incr/decr){
2. //code to be executed
3. }

Flowchart of for loop in C



C for loop Examples

Let's see the simple program of for loop that prints table of 1.

1. #include<stdio.h>
2. **int** main(){
3. **int** i=0;

```
4. for(i=1;i<=10;i++){
5. printf("%d \n",i);
6. }
7. return 0;
8. }
```

Infinite Loop in C

An infinite loop is a looping construct that does not terminate the loop and executes the loop forever. It is also called an **indefinite** loop or an **endless** loop. It either produces a continuous output or no output.

When to use an infinite loop

An infinite loop is useful for those applications that accept the user input and generate the output continuously until the user exits from the application manually. In the following situations, this type of loop can be used:

- All the operating systems run in an infinite loop as it does not exist after performing some task. It comes out of an infinite loop only when the user manually shuts down the system.
- All the servers run in an infinite loop as the server responds to all the client requests. It comes out of an indefinite loop only when the administrator shuts down the server manually.
- All the games also run in an infinite loop. The game will accept the user requests until the user exits from the game.

Example of Infinite loop

```
1. #include <stdio.h>
2. int main()
3. {
4.    for(;;)
5.    {
6.      printf("Hello javatpoint");
7.    }
8. return 0;
9. }
```

```
1. #include <stdio.h>
2. int main()
3. {
4.   int i=0;
5.   while(1)
6.   {
7.       i++;
8.       printf("i is :%d",i);
9.   }
10. return 0;
11. }
```